WEBEX EMBEDDED APP WORKSHOP

Build a team retro board

A student guide to exploring a real codebase, fixing bugs, and creating an app that runs inside Webex.

FOUR TASKS · ONE TEAM PROJECT · YOUR OWN APP AT THE END

What you are building

A **retrospective** is a team meeting to reflect on how work is going. This project gives your team a shared board with three columns: **Start**, **Stop**, and **Continue**. Add cards, vote on ideas, and discuss what to improve.

You will work with HTML, CSS, JavaScript, a small Node.js server, and the Webex Embedded Apps SDK. The project follows a typical engineering journey: get an existing app running, understand it, improve it, then design something of your own.

Your route through the workshop

01 GET STARTED Run Hello Retro locally and open it in Webex.	**02 EXPLORE** Trace how events, state, and rendering fit together.
03 IMPROVE Investigate behaviour, fix bugs, and add features.	**04 CREATE** Build an embedded app idea and present your work.

What you need

Install **Git**, **Node.js (LTS)**, and a code editor such as VS Code. You will also need a GitHub account, **cloudflared** for a temporary HTTPS tunnel, and access to a **Webex Developer Sandbox**. A standard free Webex account cannot create embedded apps.

Install links: Git · Node.js · VS Code · cloudflared

Work in teams: one person can host the shared board. Each student can still run a separate local copy for development.

Task 1 / Set up and run Hello Retro

Start by getting the project running on your computer. You will run the app locally for quick feedback, then use a temporary public URL so Webex can load it.

1. Fork and clone the project

Use the **Fork** button on the project's GitHub page. Clone your copy, then enter the project folder:

```
git clone https://github.com/<your-username>/webex-retro-board-project.git
cd webex-retro-board-project
ls
```

The main app files are `index.html`, `styles.css`, and `app.js`.

2. Install dependencies and start the app

```
npm install
npm start
```

Open `http://localhost:3000`. Look for **Shared board connected**. The server stores the board in `data/board.json`, so it survives a restart. Run `npm install` once; run `npm start` whenever you work on the project.

A warning that the Webex SDK is unavailable in a normal browser is expected. The rest of the app still works.

3. Set up a Developer Sandbox

Embedded apps need a licensed Webex organisation. One person on your team should request a free sandbox from the Developer Sandbox guide. Use an email address that is not already registered with Webex; a plus alias can help. The sandbox admin adds teammates in Control Hub (up to 10 people). Everyone signs into Webex with their sandbox account.

4. Give Webex a temporary public address

Keep `npm start` running. Open a second Terminal window and start a Quick Tunnel:

```
cloudflared tunnel --url http://localhost:3000
```

When a URL appears, open it in a browser and check that the board loads. Keep both Terminal windows open. The tunnel URL is temporary and may change when restarted.

Shared team board: One person runs the Node server and tunnel, then shares the URL. Everyone visiting it sees the same live board. The host computer must stay on. Anyone with the URL can change the trial board, so use demo data only and stop the tunnel when finished.

Task 1 / Open the app in Webex

5. Create an Embedded App

Sign in to developer.webex.com with your sandbox account. Open **My Webex Apps** → **Create a New App** → **Create Embedded App**, then enter:

- **Where does your app work?** Select Meeting and Messaging.
- **App name:** for example, Team Retro Board.
- **Start Page URL:** your current tunnel URL plus `/index.html`.
- **Valid domains:** only the tunnel host, without `https://` or a path.

Example: if your tunnel is `https://some-words.trycloudflare.com`, use `https://some-words.trycloudflare.com/index.html` as the Start Page URL and `some-words.trycloudflare.com` as the valid domain.

6. Launch it

- In the Webex desktop app, start a meeting or open a space.
- Open the Apps area and find your in-development app.
- Choose **Open and share my personal information**. The greeting should include your name.

If it does not load
First open the tunnel URL in a browser. Check that the Node server and tunnel are running, the Start Page URL ends in `/index.html`, and the valid domain matches the current tunnel host. If the tunnel restarted, update the app settings. Make sure you are signed into the sandbox account.

Static demo option: GitHub Pages

For a front-end-only demo, deploy the `main` branch from the repository root using **Settings** → **Pages**. Use the Pages URL plus `/index.html` as the Start Page URL, and its host as the valid domain. Pages has a stable URL, but changes appear after pushing and deployment; without the Node backend, each browser stores its own board.

The name says ‘Hi there’?

That is expected in a regular browser. Inside Webex, the app needs permission to access your name. Reopen it and grant the personal information permission; if needed, close and reopen the app because embedded apps can cache content.

Keep your local server running while you develop. Most changes can be tried in a browser at `http://localhost:3000`.

Task 2 / Explore the code

Open `app.js`. Follow one action from the user interface through the code and back to the screen.

Four areas to know

WEBEX SDK HELPER Gets user details, applies the theme, and shares the app.	**STATE** Stores the board and handles add, delete, vote, save, and load.
RENDER Turns board data into cards, columns, and counts on screen.	**EVENTS** Connects clicks and form submissions to state changes and rendering.

Example: adding a card

The form event in `wireEvents()` calls `addCard()`. That updates the board state, then `renderBoard()` draws the updated board.

Explore the shared board

- Add a card to Start, Stop, and Continue.
- Vote on a teammate's card and delete a card.
- Open the same host URL on another screen. Do the changes appear there too?

Debug with a plan

- Reproduce the issue and note what you expected versus what happened.
- Use the browser developer tools: F12 or Ctrl+Shift+I (Windows/Linux), Cmd+Option+I (Mac). In Webex, right-click inside the embedded app and choose Inspect if available.
- Check the Console for errors. The SDK warning in a normal browser is expected.
- Use Sources to set a breakpoint in `app.js`; inspect Variables and Call Stack while stepping through the action.
- Use Network to check `GET /api/board` and the `/ws` WebSocket. Use Elements to inspect layout.

VS Code debugging

Open the project folder and choose a configuration under **Run and Debug**, then press F5. **Debug Node backend** runs `server.js` (stop any server already on port 3000 first). **Debug Retro Board in Chrome** uses the running local server; set breakpoints in `app.js`.

Temporary logs can help you follow the flow. Add a small `console.log()` near the event, state change, and render, then remove it when you are done. Backend messages appear in the terminal running `npm start`.

Task 3 / Fix and extend the board

Before changing the code, try the board as a group. Make a prediction, test it, then agree on what behaviour would be better.

Part A: investigate

- What happens if one person votes on the same card several times?
- What happens to the five-minute timer after a page refresh?
- What if two people start a timer on their own screens?
- What happens if you delete a card someone else added?

Exercise: make votes count once per person

Change `voteCard` so a person can vote only once on each card. The app will need to remember who voted, not only a number of votes. Inspect how `card.votes` is represented in the State area.

Part B: add features

1. Sort cards by votes

Show the most-voted cards first in each column. When votes change, the card order should update. Look at `renderColumn()`; sorting can happen when cards are drawn.

2. Let people delete their own cards

Use `card.author` and `webexHelper.getUserName()` to check who created a card. Update the delete action and consider hiding the delete button on cards belonging to others.

3. Add a facilitator action to complete the retro

Add a **Complete retro** button that summarizes the top-voted card or tied cards in each column, then clears the board for everyone. Show it only to the meeting host or co-host.

- Find the current role using the Webex SDK `meeting:roleChanged` event.
- Find the top card(s) in each column from the board state.
- Add a clear action alongside add, delete, and vote. In shared mode, update `server.js` `applyAction` too.
- Use the role to show or hide the button. Discuss why a browser-only role check is not production security.

Bonus: make the timer shared

Move timer data into the shared board so everyone sees the same countdown and refreshes do not reset it. Store an end timestamp rather than only a remaining-seconds number; clients can calculate the remaining time locally. Add an action so timer changes are broadcast by the server.

Stretch challenge: post the summary to the Webex space

Use the Webex Messages API to post the completion summary. This needs an OAuth Integration with `spark:messages_write`, the space ID, and a server-side token exchange. Read the Messages API guide and plan the authorization flow before coding.

Safe team demo
The trial backend has no login. The role check is a user interface guard, not authorization enforced by the server. Keep the tunnel temporary and use demo data.

Task 4 / Make it your own

Use what you have learned to design a new embedded app. Pick a small problem your team could solve during a meeting.

Ideas to get you started

- Planning poker for estimating work items.
- A stand-up board for yesterday, today, and blockers.
- A team icebreaker or quiz to start a meeting.

Explore more ideas in the Webex App Hub Embedded Apps.

Prepare your presentation

- Demo your new embedded app running inside Webex.
- Explain one issue you found in the retro board and how you tracked it down.
- Show a feature you added and explain the choices you made.

Team check-in

[ ]	The retro board runs at localhost:3000.
[ ]	The team can open the same shared board URL.
[ ]	We traced an action through events, state, and render.
[ ]	We tested and discussed the behaviour we changed.
[ ]	We can demo our own app and describe our decisions.

The engineering loop
Observe -> make a prediction -> trace the code -> try a change -> check the result -> explain what you learned.

Project resources

Request a Developer Sandbox · Webex Developer Portal · Messages API

Workshop source: Webex Retro Board Project README. For project-specific instructions and the latest code, use the repository README.